```
select avg (balance)
from account
```

We use the aggregate function **count** frequently to count the number of tuples in a relation. The notation for this function in SQL is **count** (*). Thus, to find the number of tuples in the *customer* relation, we write

```
select count (*)
from customer
```

SQL does not allow the use of **distinct** with **count**(*). It is legal to use **distinct** with **max** and **min**, even though the result does not change. We can use the keyword **all** in place of **distinct** to specify duplicate retention, but, since **all** is the default, there is no need to do so.

If a **where** clause and a **having** clause appear in the same query, SQL applies the predicate in the **where** clause first. Tuples satisfying the **where** predicate are then placed into groups by the **group by** clause. SQL then applies the **having** clause, if it is present, to each group; it removes the groups that do not satisfy the **having** clause predicate. The **select** clause uses the remaining groups to generate tuples of the result of the query.

To illustrate the use of both a **having** clause and a **where** clause in the same query, we consider the query "Find the average balance for each customer who lives in Harrison and has at least three accounts."

```
select depositor.customer-name, avg (balance)
from depositor, account, customer
where depositor.account-number = account.account-number and
      depositor.customer-name = customer.customer-name and
      customer-city = 'Harrison'
group by depositor.customer-name
having count (distinct depositor.account-number) >= 3
```

4.5 Null Values

SQL allows the use of *null* values to indicate absence of information about the value of an attribute.

We can use the special keyword **null** in a predicate to test for a null value. Thus, to find all loan numbers that appear in the *loan* relation with null values for *amount*, we write

```
select loan-number
from loan
where amount is null
```

The predicate **is not null** tests for the absence of a null value.

The use of a *null* value in arithmetic and comparison operations causes several complications. In Section 3.3.4 we saw how null values are handled in the relational algebra. We now outline how SQL handles null values.